

**Write a C program to implement Fractional knapsack problem using Greedy method.**

**Write a C program to implement Prim's algorithm to generate a minimum cost spanning tree.**

```
#include <stdio.h>
int n, cost[10][10];
void prim();
void main()
{
    int i, j;
    printf("\nEnter the no. of vertices:");
    scanf("%d", &n);
    printf("\nEnter the cost matrix:\nEnter 999 if there is no edge between the
vertices\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
        }
    prim();
}
void prim()
{
    int i, j, startVertex, endVertex;
    int k, nearr[10], temp, minimumCost = 0, tree[10][2];

    /*Initialize nearrarray to -1
    for (i = 0; i < n; i++)
        nearr[i] = -1;

    /*For first smallest edge */
    temp = cost[0][0];
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (cost[i][j] < temp)
            {
                temp = cost[i][j];
                startVertex = i;
                endVertex = j;
            }
        }
    }
}
```

```

    }
}
/* Now we have first smallest edge in graph */
tree[0][0] = startVertex;
tree[0][1] = endVertex;
minimumCost = cost[startVertex][endVertex];

/* To indicate included vertices initialize nearr[] for them to 100 */
nearr[startVertex] = 100;
nearr[endVertex] = 100;

/* Now initialize nearr[] */

for (i = 0; i < n; i++)
{
    if(nearr[i] != 100)
        if(cost[i][startVertex] < cost[i][endVertex])
            nearr[i] = startVertex;
        else
            nearr[i] = endVertex;
}

/* Now find out remaining n-2 edges */
temp = 999;
for (i = 1; i < n - 1; i++) // outer for loop starts
{
    for (j = 0; j < n; j++)
    {
        if (nearr[j] != 100 && cost[j][nearr[j]] < temp)
        {
            temp = cost[j][nearr[j]];
            k = j;
        }
    }

    /* Now we have got next smallest edge */
    tree[i][0] = k;
    tree[i][1] = nearr[k];
    minimumCost = minimumCost + cost[k][nearr[k]];
    nearr[k] = 100;
    /* Now find if k is nearest to any vertex than its previous near
value */

```

```

    for (j = 0; j < n; j++)
    {
        if (nearr[j] != 100 && cost[j][nearr[j]] > cost[j][k])
            nearr[j] = k;
    }
    temp = 999;
} /* outer for loop ends */
/* Now we have the answer, just print it */
printf("\nThe min spanning tree is \n");
for (i = 0; i < n - 1; i++)
{
    for (j = 0; j < 2; j++)
        printf("%d ", tree[i][j]);
    printf("\n");
}
printf("\nMin cost: %d", minimumCost);
}

```

### Output:

Enter the no. of vertices: 9

Enter the cost matrix:

Enter 999 if there is no edge between the vertices

```

999 1 999 999 999 999 999 8 999
1 999 8 999 999 999 999 11 999
999 8 999 7 999 4 999 999 2
999 999 7 999 9 14 999 999 999
999 999 999 9 999 10 999 999 999
999 999 4 14 10 999 2 999 999
999 999 999 999 999 2 999 4 16
8 11 999 999 999 999 4 999 7
999 999 2 999 999 999 16 7 999

```

The min spanning tree is

```

0 1
2 1
8 2
5 2
6 5
7 6
3 2
4 3

```

Min cost: 37

